Mock Expectations

A mock object retains the basic idea of the stub—returning a specified value without actually calling a live method—and adds the requirement that the specified method must be called during the test. A mock is like a stub with attitude, expecting—nay, demanding—that its parameters be matched in the test or else you get a test failure.

In RSpec, you use the expect method to create mock expectations. This can be applied to full or partial doubles:

```
it "expects stuff" do
  mocky = double("Mock")
  expect(mocky).to receive(:name).and_return("Paul")
  expect(mocky).to receive(:weight).and_return(100)
  expect(mocky.name).to eq("Paul")
end
```

This test fails:

```
Failures:
  1) Project expects stuff
     Failure/Error: expect(mocky).to receive(:weight).and_return(100)
       (Double "Mock").weight(any args)
           expected: 1 time with any arguments
           received: 0 times with any arguments
```

The test sets up two mock expectations, mocky.name and mocky.weight, but only one of those two mocked methods is called in the test. Hence, it's an unsatisfied expectation. To make the test pass, add a call to mocky.weight:

```
it "expects stuff" do
  mocky = double("Mock")
  expect(mocky).to receive(:name).and_return("Paul")
  expect(mocky).to receive(:weight).and_return(100)
  expect(mocky.name).to eq("Paul")
  expect(mocky.weight).to eq(100)
end
```

This also works for existing objects:

mocks/01/spec/models/project_spec.rb

```
it "mocks an object" do
  mock_project = Project.new(name: "Project Greenlight")
  expect(mock_project).to receive(:name).and_return("Fred")
  expect(mock_project.name).to eq("Fred")
end
```